# **Specification of Python §1**—2026 edition

Martin Henz, Wang Yuyang, Lucas Ng, Jonas Ong

National University of Singapore
School of Computing

October 17, 2026

The language "Python §$x$" is a sublanguage of Python 3.13 and defined in the documents titled "Python §$x$", where $x$ refers to the respective textbook chapter.

## 1 Syntax

A Python program is a *program*, defined using Backus-Naur Form[1] as follows:

[1]We adopt Henry Ledgard's BNF variant that he described in *A human engineered variant of BNF*, ACM SIGPLAN Notices, Volume 15 Issue 10, October 1980, Pages 57-62. In our grammars, we use **`bold`** font for keywords, *italics* for syntactic variables, $\epsilon$ for nothing, $x \mid y$ for $x$ or $y$, [ $x$ ] for an optional $x$, $x$... for zero or more repetitions of $x$, and ( $x$ ) for clarifying the structure of BNF expressions.

| | | | |
|---|---|---|---|
| *program* | ::= | *import-stmt*... *block* | program |
| *import-stmt* | ::= | **from** *dotted-name* **import** *import-clause* | import statement |
| *dotted-name* | ::= | *name* ( **.** *name* )... | dotted identifier chain |
| *import-clause* | ::= | *import-as-names* | |
| | \| | **(** *import-as-names* **)** | wildcard or name list |
| *import-as-names* | ::= | *import-as-name* ( **,** *import-as-name* )... | clause name list |
| *import-as-name* | ::= | *name* [ **as** *name* ] | clause name and alias |
| *statement* | ::= | *name* **=** *expression* | assignment |
| | \| | **def** *name* **(** *names* **) :** *block* | function definition |
| | \| | **return** *expression* | return statement |
| | \| | *if-statement* | conditional statement |
| | \| | *expression* | expression statement |
| *names* | ::= | $\epsilon$ \| *name* ( **,** *name* )... | name list |
| *if-statement* | ::= | **if** *expression* **:** *block* | |
| | | ( **elif** *expression* **:** *block* )... | |
| | | **else :** *block* | conditional statement |
| *block* | ::= | *statement*... | block statement |
| *expression* | ::= | *integer* | primitive integer expression |
| | \| | *float* | primitive float expression |
| | \| | *complex* | primitive complex expression |
| | \| | **True** \| **False** | primitive boolean expression |
| | \| | **None** | primitive None expression |
| | \| | *string* | primitive string expression |
| | \| | *name* | name expression |
| | \| | *expression binary-operator expression* | binary operator combination |
| | \| | *unary-operator expression* | unary operator combination |
| | \| | *expression binary-logical expression* | logical composition |
| | \| | *expression* **(** *expressions* **)** | function application |
| | \| | **lambda** (*name* \| *names*) **:** *expression* | lambda expression |
| | \| | *expression* **if** *expression* **else** *expression* | conditional expression |
| | \| | **(** *expression* **)** | parenthesised expression |
| *binary-operator* | ::= | **+** \| **-** \| ***** \| **/** \| ****** \| **%** | |
| | \| | **==** \| **!=** \| **>** \| **<** \| **>=** \| **<=** | binary operator |
| *unary-operator* | ::= | **not** \| **-** \| **+** | unary operator |
| *binary-logical* | ::= | **and** \| **or** | logical composition symbol |
| *expressions* | ::= | $\epsilon$ \| *expression* ( **,** *expression* )... | argument expressions |

## Indentation

In Python, indentation is syntactically significant and strictly enforces code block structure. Unlike languages using braces, Python employs whitespace (spaces or tabs) to delimit blocks for control flow (e.g., `if`, `for`, `while`), function definitions, and class declarations. Key rules:

- Consistency: Use 4 spaces per indentation level (PEP 8 recommendation). Mixing tabs and spaces is prohibited.
- Alignment: Statements within the same block must align vertically. Misaligned indentation raises an `IndentationError`.

- Nesting: Each nested block increases indentation by one level. Dedenting resumes the outer block.
- Line Continuation: For multi-line statements, align wrapped lines with the opening delimiter or use an extra level.

## Operator Precedence and Associativity

When an expression combines several operators, *precedence* decides which operator binds first, and *associativity* decides the order in which two operators of the same precedence group are applied. The table below lists the operators, function application, the conditional expression (**`if` / `else`**), and **`lambda`** from the grammar above, in decreasing order of precedence: an entry binds more tightly than every entry below it. Use parentheses to override the default grouping.

| Operators | Description | Associativity |
|---|---|---|
| **`( )`** | function application | left to right |
| **`**`** | exponentiation | right to left |
| **`+ -`** | unary plus, unary minus | n/a |
| **`* / %`** | multiplication, division, modulo | left to right |
| **`+ -`** | addition, subtraction | left to right |
| **`< > == != >= <=`** | comparison | n/a |
| **`not`** | logical negation | n/a |
| **`and`** | logical conjunction | left to right |
| **`or`** | logical disjunction | left to right |
| **`if / else`** | conditional expression | right to left |
| **`lambda`** | lambda expression | n/a |

Notes:

- Since **`**`** binds more tightly than unary **`+/-`**, the expression **`-2 ** 2`** is **`-(2 ** 2)`**, not **`(-2) ** 2`**.
- Comparison operators are non-associative: unlike full Python, where **`a < b < c`** is equivalent to **`a < b and b < c`** (evaluating **`b`** once and short-circuiting before **`c`** is reached), the sublanguages give every comparison exactly two operands. Chaining, as in **`a < b < c`**, is therefore rejected outright with an error that says that chained comparisons are not supported — a deliberate sublanguage restriction, rather than giving chained comparisons some other, non-standard meaning.
- Unary **`+/-`**, **`not`** and **`lambda`** take a single operand to their right, so stacking two of them has only one possible grouping (e.g. **`- -x`** always means **`-(-x)`**); associativity does not apply and is marked "n/a" for a different reason than for comparison operators above: here nothing is excluded, there is simply no choice to make.
- The conditional expression and **`lambda`** have the lowest precedence and extend as far to the right as possible; a **`lambda`** body or the **`else`** branch of a conditional expression may itself be another conditional expression or **`lambda`**.

## Restrictions

- Re-declaration variables or functions is not allowed. Once a variable or function is defined, it cannot be redefined with the same name in the same scope [2].

[2] Scope refers to the region of a program in which a particular name (such as a variable, function, or class) is defined and can be accessed. In other words, it determines the part of the program where you can use that name without causing a name error. Scope is determined by the program's structure (usually its lexical or textual layout) and governs the visibility and lifetime of variables and other identifiers.

In Python, the *scope* of a declaration is determined lexically: a variable declared inside a function is local to that function; if it is declared outside any function, it is global (i.e., module-level). Moreover, if a variable is declared in an enclosing function, it is available to inner functions (the enclosing scope), and if not found in these scopes, Python looks into the built-in scope.

Reassignment is not allowed. Within any scope, each name (including function parameters, variables, and function names) must be declared or defined at most once. Therefore, a parameter or declared variable must not appear on the left-hand side of an assignment, and a name must not be defined by multiple assignments or function definitions in the same scope.

## Names

Names[3] start with _, or a letter[4] and contain only _, letters or digits[5]. Reserved words[6] are not allowed as names.
Valid names are `x`, `_45`, and $\pi$, but always keep in mind that programming is communicating and that the familiarity of the audience with the characters used in names is an important aspect of program readability.

## Numbers

Python supports three numeric types: integers (`int`), floats (`float`), and complex numbers (`complex`).
A numeric literal never includes a leading sign: a `+` or `-` written before a number, as in `-2` or `-3.14`, is unary-operator application (see "Operator Precedence and Associativity" above), not part of the literal itself. This is why **`-2 ** 2`** is **`-(2 ** 2)`** (i.e. $-4$) rather than **`(-2) ** 2`** (i.e. $4$): the sign is applied only after **`**`** has already bound the unsigned literal **`2`** to its exponent.

### Integers

Integers can be represented in decimal notation. Additional base notations are supported, such as binary (`0b1010` or `0B1010`), octal (`0o777` or `0O777`), and hexadecimal (`0x1A3F` or `0X1A3F`). Underscores (_) may be used for readability (e.g., `1_000_000`). Examples include `42`, `0b1101` (and, with a unary minus applied, `-0b1101`), and `0x_FF_00`.

### Floats

Floats use decimal notation with an optional decimal dot. Scientific notation (multiplying the number by $10^x$) is indicated with the letter `e` or `E`, followed by the exponent `x`. Special values inf (infinity), -inf, and nan (not a number) are allowed. Examples include `3.14`, `0.001e+05` (and, with a unary minus applied, `-0.001e+05`), and `6.022E23`.

### Complex Numbers

A complex value can be formed from a real part and an imaginary part, written `<real>±<imag>j`, where `j` (or `J`) denotes the imaginary unit. Both parts are stored as floats. The imaginary part is mandatory (e.g., `5j`, `0j`, and `0+3j` is valid, `5` alone is real). As with the sign of a real number, the `±` between the real and imaginary parts is not part of a single token: only `<imag>j` (e.g., `3j`) is a numeric literal (an *imaginary literal*), so `2+3j` parses as the real literal `2` combined with the imaginary literal `3j` by binary `+`, and `-4.5j` as unary `-` applied to the imaginary literal `4.5j`. Examples include `2+3j`, `-4.5J`, `0j`, and `1e3-6.2E2J`.

## Strings

Strings are of the form `"`*double-quote-characters*`"`, where *double-quote-characters* is a possibly empty sequence of characters without the character `"` and without the newline character; of the form `'`*single-quote-characters*`'`, where *single-quote-characters* is a possibly empty sequence of characters without the character `'` and without the newline character; and of the form `'''`*triple-single-quote-characters*`'''`, or `"""`*triple-double-quote-characters*`"""`, where

[3] In Python 3.13 Documentation, these names are called *identifiers*.
[4] By *letter* we mean Unicode letters (L) or letter numbers (Nl).
[5] By *digit* we mean characters in the Unicode categories Nd (including the decimal digits 0, 1, 2, 3, 4, 5, 6, 7, 8, 9), Mn, Mc and Pc.
[6] By *reserved word* we mean any of: **`False`**, **`await`**, **`else`**, **`import`**, **`pass`**, **`None`**, **`break`**, **`except`**, **`in`**, **`raise`**, **`True`**, **`class`**, **`finally`**, **`is`**, **`return`**, **`and`**, **`continue`**, **`for`**, **`lambda`**, **`try`**, **`as`**, **`def`**, **`from`**, **`nonlocal`**, **`while`**, **`assert`**, **`del`**, **`global`**, **`not`**, **`with`**, **`async`**, **`elif`**, **`if`**, **`or`**, **`yield`**. These are all reserved words, or keywords of the language that cannot be used as ordinary identifiers.

*triple-double-quote-characters* is a possibly empty sequence of characters and can span multiple lines and may contain both single and double quotes without escaping.
The following characters can be represented in strings[7] as given:

- `\<newline>`: Backslash followed by a newline is ignored.
- `\\`: Represents a backslash (i.e., \).
- `\'`: Represents a single quote (').
- `\"`: Represents a double quote (").
- `\a`: Represents the ASCII Bell (BEL) character.
- `\b`: Represents the ASCII Backspace (BS) character.
- `\f`: Represents the ASCII Formfeed (FF) character.
- `\n`: Represents the ASCII Linefeed (LF) character.
- `\r`: Represents the ASCII Carriage Return (CR) character.
- `\t`: Represents the ASCII Horizontal Tab (TAB) character.
- `\v`: Represents the ASCII Vertical Tab (VT) character.

# 2 Dynamic Type Checking

Expressions evaluate to integers, floats, complex numbers, boolean values, strings or function values. Implementations of Source generated error messages when unexpected values are used as follows.
Only function values can be applied using the syntax:

*expression* ::= *name* **(** *expressions* **)**

For compound functions, implementations need to check that the number of *expressions* matches the number of parameters.
The following table specifies what arguments Python's operators take and what results they return. Implementations need to check the types of arguments and generate an error message when the types do not match.

[7] In Python 3.13 Documentation, unless an 'r' or 'R' prefix is present, escape sequences in string and bytes literals are interpreted according to rules similar to those used by Standard C. *identifiers*.

| operator | argument 1 | argument 2 | result |
|---|---|---|---|
| `+,-,*,%,//` | int | int | int |
| `/` | int | int | float |
| `+,-,*,/,%,//` | int | float | float |
| `+,-,*,/,%,//` | float | int, float | float |
| `+,-,*,/` | int | complex | complex |
| `+,-,*,/` | float | complex | complex |
| `+,-,*,/` | complex | int, float, complex | complex |
| `+` | string | string | string |
| `**` | int | int >= 0 | int |
| `**` | int | int < 0 | float |
| `**` | int | float | float |
| `**` | int | complex | complex |
| `**` | float | int, float | float |
| `**` | float | complex | complex |
| `**` | complex | int, float, complex | complex |
| `==,!=` | bool,function | any | error |
| `==,!=` | any | bool,function | error |
| `==,!=` | anything else | anything else | bool |
| `>,>=,<,<=` | int,float | int,float | bool |
| `>,>=,<,<=` | string | string | bool |
| `and` | bool | any | any |
| `or` | bool | any | any |
| `not` | bool | | bool |
| `-` | int | | int |
| `-` | float | | float |
| `-` | complex | | complex |

Following **`if`** and **`elif`**, Python §$x$ only allows boolean expressions. Lambda expressions can only be used as arguments to operators when they are parenthesized. The operators **`==`** and **`!=`** follow the Python 3.14 Documentation except that they must not be applied to functions and boolean values.

# 3 Standard Library

The standard library contains constants and functions that are always available in this language. In py-slang, the standard library functions are not implemented using a conventional split between primitive and predeclared functions. Unlike in Source, where predeclared functions are defined using built-in primitives, all standard library functions in py-slang are written directly in TypeScript and embedded into the runtime of the CSE Machine. These functions are treated as part of the host environment rather than as user-level definitions, and they interact directly with the evaluator's internal control stack and environment. This design simplifies the interpreter architecture and allows for tighter integration with the execution model, enabling better support for visualization, error handling, and platform-specific extensions within the Source Academy learning environment.
The standard library for Python §$x$ is documented in online documentation of Python §$x$.

## MISC Library

The following names are provided by the MISC library:

- `str(x)`: *primitive*, converts the value `x` to a complex number, or a string representation of `x` respectively, and returns the respective value.

- `repr(s)`: *primitive*, returns a string representation of `s` that is unambiguous and suitable for debugging; for many values, this string can be used to reconstruct `s`.
- `is_boolean(x)`, `is_integer(x)`, `is_float(x)`, `is_string(x)`, `is_none(x)`, `is_function(x)`, `is_complex(x)`: *primitive*, returns `True` if the type of `x` matches the function name and `False` if it does not.
- `complex(r, i)`: *primitive*, takes in two numbers `r` and `i`, and returns the complex number with real part `r` and imaginary part `i`.
- `real(x)`, `imag(x)`: *primitive*, return the real and imaginary parts of a complex number `x` respectively.
- `input(s)`: *primitive*, pops up a window that displays the *string* `s`, provides an input line for the user to enter a text, a "Cancel" button and an "OK" button. The call of `input` suspends execution of the program until one of the two buttons is pressed. If the "OK" button is pressed, `input` returns the entered text as a string. If the "Cancel" button is pressed, `input` returns a non-string value.
- `min(x, y, *args)` and `max(x, y, *args)`: *primitive*, takes in two or more arguments and returns the smallest and largest of the arguments respectively. If multiple arguments are equally small or large, the first one encountered in the argument list is returned. The arguments must be mutually comparable (i.e., they must support the < and > operators).
- `abs(x)`: *primitive*, returns the absolute value of `x`. For an `int` input, it returns the non-negative integer equivalent. For a `float` input, it returns the positive floating-point number representing its magnitude. For a `complex` input, it returns the modulus of the complex number, which is a non-negative float.
- `print(*x)`: *primitive*, takes in any number of parameters, converts them to their string representations using `str` and writes them to the console, followed by a newline character.
- `error(*x)`: *primitive*, takes in any number of parameters, converts them to their string representations using `str` and writes them to the console, followed by a newline character. The evaluation of any call of `error` aborts the running program immediately.
- `breakpoint()`: *primitive*, evaluates to the `None` value with no effect on the program. In the stepper and CSE Machine visualization, it additionally marks a breakpoint that the breakpoint navigation can jump to.
- `arity(f)`: *primitive*, takes in a function `f` and returns the minimum number of parameters that `f` can accept.
- `round(x)`[8]: *primitive*, takes in an int or float `x` and returns the nearest integer to `x`.
- `round(x, n)`: *primitive*, takes in an int or float `x` and an int `n`, and returns the value of `x` rounded to `n` digits of precision after the decimal point. If the second argument `n` is None, it is treated as `round(x)`.
- `len(s)`: *primitive*, takes in a container for a finite number of values (e.g., a string), and returns the number of elements in the container.
- `time_time()`: *primitive*, returns number of milliseconds elapsed since January 1, 1970 00:00:00 UTC
- `random_random()`: *primitive*, returns a random float uniformly in the range $[0.0, 1.0)$.

All library functions can be assumed to run in $O(1)$ time, except `print`, `error`, `str` and `repr`, which run in $O(n)$ time, where $n$ is the size (number of components such as pairs) of their arguments.

[8] In Python 3.13, when the value to be rounded is exactly halfway between two possible rounded values, it rounds to the nearest even number. For example, `round(2.5)` would return 2, while `round(3.5)` would return 4.

### MATH Library

The following names are provided by the MATH library:

- `math_`*name*, where *name* is any name specified in the Python **`math`** library, see Python 3.14 Documentation. Examples:
  - `math_nan`: *primitive*, refers to the NaN ("Not a Number") value,
  - `math_inf`: *primitive*, refers to the Infinity value,
  - `math_pi`: *primitive*, refers to the mathematical constant $\pi$,
  - `math_sqrt(n)`: *primitive*, returns the square root of the *number* `n`.

All functions can be assumed to run in $O(1)$ time and are considered *primitive*.

## Deviations from native Python

We intend the Python §$x$ to be a conservative extension of native Python: Every correct Python §$x$ program should behave *exactly* the same using a Python §$x$ implementation, as it does using a native Python implementation. We assume, of course, that suitable libraries are used by the TypeScript implementation, to account for the predefined names of Python §$x$. This section lists some exceptions where we think a Python §$x$ implementation should be allowed to deviate from the native Python specification, for the sake of internal consistency and esthetics.

**Output Behavior Differences Between py-slang and Standard Python REPL:** Python provides an interactive mode, where users can type and immediately evaluate Python expressions line by line. This mode is entered when you simply type python in a terminal without specifying a file. This is commonly referred to as the Python REPL. In the standard Python REPL, any evaluated expression automatically has its result printed to the console, even if the user does not explicitly call `print()`. For example:

```
>>> 1 + 2
3
>>> "hello"
'hello'
```

In contrast, py-slang implements the behavior of **non-interactive** Python, the mode used when you pass a Python program to `python`. Only expressions explicitly passed to `print()` produce output. If the expression is evaluated without a print call, no output will appear, even though a value is computed.

For example, in py-slang:

```
print(1 + 2)     # Outputs: 3
```

But

```
1 + 2            # Outputs nothing
```